

# tiny-llm: Week 1 Day 3–5

Grouped Query Attention · RMSNorm 与 MLP · Qwen3 模型

中英双语对照 · Bilingual Edition

## Day 3: Grouped Query Attention (GQA)

On Day 3, we will implement grouped-query attention (GQA). Qwen3 uses GQA to reduce the computational and memory costs of the key (K) and value (V) projections. In multi-head attention (MHA), every query (Q) head has a corresponding K and V head. With GQA, groups of Q heads share K and V heads. Multi-query attention (MQA) is the special case in which every Q head shares a single K/V head pair.

第 3 天，我们将实现分组查询注意力（Grouped-Query Attention, GQA）。Qwen3 使用 GQA 来降低键（K）与值（V）投影的计算与内存开销。在多头注意力（MHA）中，每个查询（Q）头都有与之对应的 K 头和 V 头；而在 GQA 中，若干个 Q 头组成一组，共享同一组 K/V 头。多查询注意力（MQA）是 GQA 的特例：所有 Q 头共享唯一的一对 K/V 头。

### 延伸阅读 / Readings

- [GQA paper](#)
- [Qwen3 layers in mlx-lm](#)
- [PyTorch scaled dot-product attention with enable\\_gqa=True](#)
- `torch.tune.modules.MultiHeadAttention`

## Task 1: Implement scaled\_dot\_product\_attention\_grouped

### 任务 1：实现 scaled\_dot\_product\_attention\_grouped

You will need to modify the following file:

你需要修改以下文件：

```
src/tiny_llm/attention.py
```

In this task, we will implement grouped scaled dot-product attention, which forms the core of GQA.

本任务将实现“分组缩放点积注意力”，它是 GQA 的核心。

Implement `scaled_dot_product_attention_grouped` in `src/tiny_llm/attention.py`. It is similar to standard scaled dot-product attention, but it supports a number of query heads that is a multiple of the number of key/value heads.

请在 `src/tiny_llm/attention.py` 中实现 `scaled_dot_product_attention_grouped`。它与标准的缩放点积注意力类似，区别在于：查询头的数量可以是键/值头数量的整数倍。

The main process is the same as standard scaled dot-product attention. The difference is that K and V heads are shared across multiple Q heads. Instead of  $H_q$  separate K and V heads, there are  $H$  K and V heads, each shared by  $n\_repeats = H_q // H$  query heads.

整体流程与标准缩放点积注意力一致，区别在于 K/V 头被多个 Q 头共享：不存在  $H_q$  个彼此独立的 K/V 头，而是只有  $H$  个 K/V 头，每个被  $n\_repeats = H_q // H$  个查询头共享。

Reshape query, key, and value so that K and V can be broadcast to the query heads in their respective groups during the matrix multiplications.

请调整 query、key、value 的形状，使 K 和 V 能在矩阵乘法中广播到各自分组内的所有查询头上。

- Separate the  $H$  and  $n\_repeats$  dimensions in query.
- Add a dimension of size 1 for  $n\_repeats$  in key and value so that they broadcast across each group.

- 在 query 中把 H 与 n\_repeats 两个维度拆开；
- 在 key 和 value 中插入一个大小为 1 的 n\_repeats 维度，使其能在每个分组内广播。

Then perform scaled dot-product attention: matrix multiplication, scaling, optional masking, softmax, and a final matrix multiplication. Broadcasting handles the head sharing without materializing repeated K and V tensors.

随后执行缩放点积注意力的标准步骤：矩阵乘法、缩放、可选的掩码、softmax，以及最后一次矩阵乘法。借助广播即可实现头共享，无需真正把 K、V 复制扩展成重复的大张量。

Using broadcasting instead of repeating K and V is more efficient because it avoids creating copies of the same data.

相比“重复复制”K 和 V，使用广播更高效，因为它避免了创建同一份数据的多个副本。

Finally, reshape the result to the expected output shape.

最后，把结果 reshape 成期望的输出形状。

```
N.. is zero or more dimensions for batches
H_q is the number of query heads
H is the number of key/value heads (H_q must be divisible by H)
L is the query sequence length
S is the key/value sequence length
D is the head dimension

query: N.. x H_q x L x D
key: N.. x H x S x D
value: N.. x H x S x D
mask: N.. x H_q x L x S
output: N.. x H_q x L x D
```

In addition to grouped heads, this function supports different query and key/value sequence lengths: Q uses length L, while K and V use length S.

除了分组头之外，该函数还支持查询与键/值使用不同的序列长度：Q 的长度为 L，而 K、V 的长度为 S。

You can test your implementation by running the following command:

可以运行以下命令测试你的实现：

```
pdm run test --week 1 --day 3 -- -k task_1
```

## Task 2: Causal Masking

### 任务 2：因果掩码

#### 延伸阅读 / Readings

- [Writing an LLM from scratch, part 9 – causal attention](#)

In this task, we will add causal masking to grouped attention.

本任务将为分组注意力加上因果掩码（causal masking）。

Causal masking prevents attention from reading future tokens. When mask is set to the string "causal", apply a causal mask.

因果掩码用于阻止注意力读取未来的 token。当 mask 被设为字符串 "causal" 时，就应用因果掩码。

The additive causal mask has shape (L, S), where L is the query sequence length and S is the key/value sequence length. Allowed positions contain 0, and masked positions contain -inf. When S is greater than L,

shift the diagonal by  $S - L$  so that the queries correspond to the final  $L$  positions in the key/value sequence. For example, if  $L = 3$  and  $S = 5$ , the mask is:

这个加性 (additive) 因果掩码的形状为  $(L, S)$ ，其中  $L$  是查询序列长度， $S$  是键/值序列长度。允许 attend 的位置取值为 0，被屏蔽的位置取值为  $-\text{inf}$ 。当  $S$  大于  $L$  时，需要把对角线偏移  $S - L$ ，使查询对应键/值序列中最后的  $L$  个位置。例如当  $L = 3$ 、 $S = 5$  时，掩码为：

```
0  0  0  -inf -inf
0  0  0   0  -inf
0  0  0   0   0
```

Implement `causal_mask` in `src/tiny_llm/attention.py`, then use it in `scaled_dot_product_attention_grouped`. Note that our shifted diagonal for  $L \neq S$  differs from the default behavior of some attention APIs.

请在 `src/tiny_llm/attention.py` 中实现 `causal_mask`，并在 `scaled_dot_product_attention_grouped` 中使用它。注意：我们在  $L \neq S$  时采用的对角线偏移方式，与某些注意力 API 的默认行为不同。

You can test your implementation by running the following command:

可以运行以下命令测试你的实现：

```
pdm run test --week 1 --day 3 -- -k task_2
```

## Task 3: Qwen3 Grouped Query Attention

### 任务 3：Qwen3 的分组查询注意力

In this task, we will implement Qwen3's grouped-query attention. Modify the following file:

本任务将实现 Qwen3 的分组查询注意力。请修改以下文件：

```
src/tiny_llm/qwen3_week1.py
```

`Qwen3MultiHeadAttention` implements attention for Qwen3. Follow this pseudocode:

`Qwen3MultiHeadAttention` 实现了 Qwen3 的注意力。请按如下伪代码实现：

```
x: B, L, E
q = linear(x, wq) -> B, L, H_q, D
k = linear(x, wk) -> B, L, H, D
v = linear(x, wv) -> B, L, H, D
q = rms_norm(q, q_norm)
k = rms_norm(k, k_norm)
q = rope(q, offset=slice(0, L))
k = rope(k, offset=slice(0, L))
(transpose as needed)
x = scaled_dot_product_attention_grouped(q, k, v, scale, mask) -> B, H_q, L, D # use
↳ float32
(transpose as needed)
x = linear(x, wo) -> B, L, E
```

Qwen3 attention has no Q/K/V projection biases, and it applies RMSNorm to each Q and K head before RoPE. We will implement the reusable RMSNorm layer on Day 4, so call `mx.fast.rms_norm` directly for `q_norm` and `k_norm` today. Use non-traditional RoPE.

Qwen3 的注意力中 Q/K/V 投影都不带偏置，并且在 RoPE 之前对每个 Q 头和 K 头分别做 RMSNorm。复用的 RMSNorm 层会在第 4 天实现，因此今天请直接调用 `mx.fast.rms_norm` 来完成 `q_norm` 和 `k_norm`。RoPE 使用非传统（non-traditional）布局。

You can test your implementation by running the following command:

可以运行以下命令测试你的实现：

```
pdm run test --week 1 --day 3 -- -k task_3
```

At the end of the day, you should be able to pass all tests of this day:

在今天结束时，你应该能通过当天的全部测试：

```
pdm run test --week 1 --day 3
```

## Day 4: RMSNorm and the Multilayer Perceptron

### 第 4 天：RMSNorm 与多层感知机

On Day 4, we will implement two important components of the Qwen3 Transformer architecture: RMSNorm and the multilayer perceptron (MLP), also known as the feed-forward network. RMSNorm is a normalization technique with less computational overhead than traditional layer normalization. The MLP applies nonlinear transformations after the attention block.

第 4 天，我们将实现 Qwen3 Transformer 架构中的两个重要组件：RMSNorm 与多层感知机（MLP，也称前馈网络）。RMSNorm 是一种归一化方法，计算开销比传统的层归一化（LayerNorm）更低。MLP 则在注意力块之后施加非线性变换。

#### Task 1: Implement RMSNorm

##### 任务 1：实现 RMSNorm

In this task, we will implement the RMSNorm layer.

本任务将实现 RMSNorm 层。

```
src/tiny_llm/layer_norm.py
```

Day 3 used `mx.fast.rms_norm` directly so that the GQA chapter could stay focused on attention. This task implements the same normalization rule as a reusable layer. From this point on, the Transformer block, final model normalization, and Q/K normalization path can use your RMSNorm implementation.

第 3 天直接调用了 `mx.fast.rms_norm`，是为了让 GQA 一章专注在注意力本身。本任务则把同一套归一化规则实现成一个可复用的层。从现在起，Transformer 块、模型最后的归一化，以及 Q/K 归一化路径，都可以改用你自己的 RMSNorm 实现。

#### 延伸阅读 / Readings

- [Root Mean Square Layer Normalization](#)
- [Qwen3 layers implementation in mlx-lm \(includes RMSNorm\)](#) - See RMSNorm.

RMSNorm is defined as:

RMSNorm 的定义如下：

$$y = \frac{x}{\sqrt{\text{mean}(x^2) + \epsilon}} \cdot \text{weight}$$

where:

其中：

- `x` is the input tensor.
  - `weight` is a learned scaling parameter.
  - `epsilon (eps)` is a small constant, such as `1e-5` or `1e-6`, added for numerical stability.
  - `mean(x^2)` is the mean of the squared elements along the final dimension.
- 
- `x` 是输入张量；
  - `weight` 是可学习的缩放参数；
  - `epsilon (eps)` 是为了数值稳定而加入的极小常数，例如 `1e-5` 或 `1e-6`；
  - `mean(x^2)` 是沿最后一个维度求得的平方元素均值。

Apply normalization independently to each feature vector along the input's final dimension. Cast the input to `float32` for the normalization calculation, including the mean, to preserve precision when the original values use `float16` or `bfloat16`. Cast the normalized value back to the input dtype before applying weight. This matches the low-precision path used by MLX's fast RMSNorm kernels: normalization statistics are accumulated in `float32`, while the final scaling happens in the model dtype.

请沿着输入的最后一个维度，对每个特征向量独立地做归一化。做归一化计算（包括求均值）时，先把输入转成 `float32`，以在原始值为 `float16` 或 `bfloat16` 时保住精度；在乘上 `weight` 之前，再把归一化后的值转回输入原始 dtype。这与 MLX 快速 RMSNorm 内核的低精度路径一致：归一化统计量在 `float32` 中累加，而最后的缩放仍在模型 dtype 下进行。

```
D is the embedding dimension.
```

```
x: N.. x D
weight: D
output: N.. x D
```

You can test your implementation by running:

可以运行以下命令测试你的实现：

```
pdm run test --week 1 --day 4 -- -k task_1
```

## Task 2: Implement the MLP Block

### 任务 2：实现 MLP 块

In this task, we will implement the MLP block named `Qwen3MLP`.

本任务将实现名为 `Qwen3MLP` 的 MLP 块。

```
src/tiny_llm/qwen3_week1.py
```

The original Transformer uses a simple position-wise feed-forward network (FFN) in each block. It consists of two linear transformations with a ReLU activation between them.

最初的 Transformer 在每个块中使用简单的逐位置前馈网络（FFN），它由两个线性变换及其之间的 ReLU 激活组成。

Modern Transformer architectures, including Qwen3, often use more advanced FFN variants. Qwen3 uses SwiGLU, a gated linear unit (GLU) variant.

包括 Qwen3 在内的现代 Transformer 架构通常使用更先进的 FFN 变体。Qwen3 使用 SwiGLU，一种门控线性单元（GLU）变体。

A plain FFN can be abstracted as:

一个普通 FFN 可以抽象为：

```
h = activation(W_up(x))
out = W_down(h)
```

A GLU keeps the same expand-then-project-back shape but adds another projection that gates the intermediate features before `W_down`. This gives the MLP a learned, input-dependent way to control which intermediate channels matter, rather than applying an activation only to the features produced by `W_up`.

GLU 保留了同样的“先升维、再投影回去”的结构，但在  $W_{\text{down}}$  之前额外加入一个投影，用来对中间特征做门控。这让 MLP 拥有一种可学习、依赖输入的方式来控制哪些中间通道更重要，而不仅仅是对  $W_{\text{up}}$  产生的特征施加一个激活函数。

SwiGLU is the GLU variant used by Qwen3:

SwiGLU 即 Qwen3 采用的 GLU 变体：

```
u = W_up(x)
g = SiLU(W_gate(x))
out = W_down(g * u)
```

### 延伸阅读 / Readings

- [Attention is All You Need \(Transformer Paper, Section 3.3 "Position-wise Feed-Forward Networks"\)](#)
- [GLU paper: Language Modeling with Gated Convolutional Networks](#)
- [SiLU \(Swish\) activation function](#)
- [SwiGLU paper: GLU Variants Improve Transformer](#)
- [PyTorch SiLU documentation](#)
- [Qwen3 layers implementation in mlx-lm \(includes MLP\)](#)

SwiGLU combines a GLU with the SiLU (sigmoid linear unit) activation function:

SwiGLU 把 GLU 与 SiLU (sigmoid linear unit) 激活函数结合起来：

- A GLU gates one linear projection of the input with another, using element-wise multiplication to control which features pass through.
- SiLU is a smooth, non-monotonic activation function. Unlike ReLU, it has no zero-gradient region across all negative inputs and can produce nonzero outputs for negative values.
- GLU 用输入的另一个线性投影去门控其中一个线性投影，通过逐元素乘法控制哪些特征得以通过；
- SiLU 是平滑且非单调的激活函数。与 ReLU 不同，它在所有负输入上并不存在梯度恒为零的区域，并且对负值也能产生非零输出。

First, implement `silu` in `basics.py`. It takes a tensor of shape  $N \times I$  and returns a tensor with the same shape:

首先，在 `basics.py` 中实现 `silu`。它接收形状为  $N \times I$  的张量，并返回同形状张量：

$$\text{SiLU}(x) = x * \text{sigmoid}(x) = \frac{x}{1 + e^{-x}}$$

Compute the sigmoid part in a numerically stable way:

请以数值稳定的方式计算 sigmoid 部分：

```
if x >= 0:
    sigmoid(x) = 1 / (1 + exp(-x))
else:
    sigmoid(x) = exp(x) / (1 + exp(x))
```

The negative branch is algebraically equivalent to the direct sigmoid formula, but it prevents  $\exp(-x)$  from becoming  $\exp(\text{large positive})$  when  $x$  is a large negative value. In vector code, first compute  $z = \exp(-\text{abs}(x))$ . Use  $z / (1 + z)$  for negative inputs and  $1 / (1 + z)$  otherwise. Do not rewrite the negative branch as  $1 - 1 / (1 + z)$ : in low precision, the fraction can round to 1, and the subtraction then incorrectly produces zero.

负半轴分支在代数上与 sigmoid 的直接公式等价，但它避免了当  $x$  为很大负数时  $\exp(-x)$  变成  $\exp(\text{很大的正数})$  而溢出。在向量化代码中，先计算  $z = \exp(-\text{abs}(x))$ ：负输入用  $z / (1 + z)$ ，其余用  $1 / (1 + z)$ 。不要把负半轴分支改写成  $1 - 1 / (1 + z)$ ：在低精度下这个分式可能舍入到 1，随后的减法就会错误地得到 0。

Then implement Qwen3MLP. Qwen3's MLP contains:

接着实现 Qwen3MLP。Qwen3 的 MLP 包含：

- A gate projection ( $W_{gate}$ )
  - An up projection ( $W_{up}$ )
  - SiLU applied to the gate projection's output
  - An element-wise product of the activated gate output and the up-projection output
  - A final down projection ( $W_{down}$ )
- 
- 门控投影 ( $W_{gate}$ )
  - 升维投影 ( $W_{up}$ )
  - 对门控投影输出施加 SiLU
  - 将激活后的门控输出与升维投影输出逐元素相乘
  - 最后的降维投影 ( $W_{down}$ )

This can be expressed as:

可以表示为：

$$\text{MLP}(x) = W_{down}(\text{SiLU}(W_{gate}(x)) \odot W_{up}(x))$$

where  $\odot$  denotes element-wise multiplication. Qwen3's MLP projections do not use biases.

其中  $\odot$  表示逐元素相乘。Qwen3 的 MLP 投影均不使用偏置。

```
N.. is zero or more dimensions for batches
E is hidden_size (embedding dimension of the model)
I is intermediate_size (dimension of the hidden layer in MLP)
L is the sequence length

input: N.. x L x E
w_gate: I x E
w_up: I x E
w_down: E x I
output: N.. x L x E
```

You can test your implementation by running:

可以运行以下命令测试你的实现：

```
pdm run test --week 1 --day 4 -- -k task_2
```

At the end of the day, you should be able to pass all tests of this day:

在今天结束时，你应该能通过当天的全部测试：

```
pdm run test --week 1 --day 4
```



## Day 5: The Qwen3 Model

### 第 5 天：Qwen3 模型

On Day 5, we will combine the components from the previous chapters into the complete Qwen3 model.

第 5 天，我们将把前几章实现的各个组件组合成完整的 Qwen3 模型。

Model-level tests require the corresponding model files. Start with the default 0.6B model; download the larger models only if you want to test them as well:

模型级别的测试需要对应的模型文件。请从默认的 0.6B 模型开始；更大的模型只在你也想测试它们时才下载：

```
hf download Qwen/Qwen3-0.6B-MLX-4bit
# Optional larger models:
hf download Qwen/Qwen3-1.7B-MLX-4bit
hf download Qwen/Qwen3-4B-MLX-4bit
```

Tests that require an unavailable model will be skipped.

需要某个未下载模型的测试会被自动跳过。

### Task 1: Implement Qwen3TransformerBlock

#### 任务 1：实现 Qwen3TransformerBlock

```
src/tiny_llm/qwen3_week1.py
```

#### 延伸阅读 / Readings

- [A Simplified Explanation of the Transformer Block](#)
- [Attention is All You Need](#)

Qwen3 uses the following Transformer block structure:

Qwen3 使用如下的 Transformer 块结构：

```
input
/ |
| input_layernorm (RMSNorm)
| |
| Qwen3MultiHeadAttention
\ |
  Add (residual)
/ |
| post_attention_layernorm (RMSNorm)
| |
| MLP
\ |
  Add (residual)
|
output
```

Run the tests for this task with:

用以下命令运行本任务的测试：

```
pdm run test --week 1 --day 5 -- -k task_1
```

## Task 2: Implement Embedding

### 任务 2：实现 Embedding

```
src/tiny_llm/embedding.py
```

#### 延伸阅读 / Readings

- [LLM Embeddings Explained: A Visual and Intuitive Guide](#)

The embedding layer maps token IDs (integers) to vectors of length `embedding_dim`. In this task, you will implement that lookup operation.

嵌入层把 token ID（整数）映射为长度为 `embedding_dim` 的向量。本任务中，你将实现这一查表操作。

```
Embedding::__call__  
weight: vocab_size x embedding_dim  
Input: N.. (tokens)  
Output: N.. x embedding_dim (vectors)
```

This can be implemented with array indexing.

这可以用数组索引来实现。

When input and output embeddings are tied, Qwen3 also uses the embedding weight as a linear projection from hidden vectors back to vocabulary logits.

当输入与输出嵌入权重绑定（**tied**）时，Qwen3 会直接复用嵌入权重，作为把隐藏向量映射回词表 logits 的线性投影。

```
Embedding::as_linear  
weight: vocab_size x embedding_dim  
Input: N.. x embedding_dim  
Output: N.. x vocab_size
```

Run the tests for this task with:

用以下命令运行本任务的测试：

```
# This task's tests use the 0.6B model and tokenizer.  
hf download Qwen/Qwen3-0.6B-MLX-4bit  
pdm run test --week 1 --day 5 -- -k task_2
```

## Task 3: Implement Qwen3ModelWeek1

### 任务 3：实现 Qwen3ModelWeek1

Now that we have built all the Qwen3 components, we can implement `Qwen3ModelWeek1`.

既然已经实现了 Qwen3 的全部组件，现在可以实现 `Qwen3ModelWeek1` 了。

```
src/tiny_llm/qwen3_week1.py
```

You will not implement the process of reading model parameters from tensor files. Instead, load the model with `mlx_lm`, then transfer its parameters into our implementation. The `Qwen3ModelWeek1` constructor therefore accepts an MLX model.

你不需要实现“从张量文件读取模型参数”的过程。取而代之，用 `mlx_lm` 加载模型，再把它的参数搬运进我们的实现中。因此 `Qwen3ModelWeek1` 的构造函数接收一个 MLX 模型对象。

The Qwen3 model has the following layers:

Qwen3 模型包含如下各层：

```
input
| (tokens: N..)
Embedding
| (N.. x hidden_size); note that hidden_size == embedding_dim
Qwen3TransformerBlock
| (N.. x hidden_size)
Qwen3TransformerBlock
| (N.. x hidden_size)
...
|
RMSNorm
| (N.. x hidden_size)
Embedding.as_linear OR linear (lm_head)
| (N.. x vocab_size)
output
```

Read the number of layers, hidden size, head dimension, and other configuration values from `mlx_model.args`, whose type is defined by `ModelArgs`. The loaded weights are available through `mlx_model.model`; use the Qwen3 implementation and model metadata to identify the corresponding layer names.

层数、hidden size、head dimension 以及其他配置值，都从 `mlx_model.args` 中读取，其类型由 `ModelArgs` 定义。加载好的权重可以通过 `mlx_model.model` 访问；请结合 Qwen3 的实现与模型元数据来确定各层对应的名称。

By this point, you have implemented `RMSNorm`. Replace the temporary Day 3 calls to `mx.fast.rms_norm` with `RMSNorm(head_dim, q_norm, eps=...)` and `RMSNorm(head_dim, k_norm, eps=...)`. They implement the same formula; the built-in calls existed only to keep the GQA chapter focused on attention.

到这里，你已经实现了 `RMSNorm`。请把第 3 天临时调用的 `mx.fast.rms_norm` 替换为 `RMSNorm(head_dim, q_norm, eps=...)` 和 `RMSNorm(head_dim, k_norm, eps=...)`。二者实现的是同一个公式；当初使用内置调用，只是为了让 GQA 一章专注在注意力上。

Different Qwen3 model variants map hidden vectors back to vocabulary logits in different ways. Some tie the input and output embeddings and use `Embedding.as_linear`; others have a separate `lm_head` linear layer. Select the strategy with `mlx_model.args.tie_word_embeddings`: if it is `True`, use `Embedding.as_linear`; otherwise, load and use `lm_head`.

不同的 Qwen3 模型变体，把隐藏向量映射回词表 logits 的方式并不相同：有的绑定输入/输出嵌入并使用 `Embedding.as_linear`，有的则带有独立的 `lm_head` 线性层。请依据 `mlx_model.args.tie_word_embeddings` 来选择策略：若为 `True`，使用 `Embedding.as_linear`；否则加载并使用 `lm_head`。

The model takes a sequence of token IDs and returns unnormalized logits for every sequence position. On Day 6, we will use the final position's logits to select the next token and generate a response.

该模型接收一串 token ID，并返回每个序列位置上的未归一化 logits (unnormalized logits)。第 6 天，我们将使用最后一个位置的 logits 来选出下一个 token，从而生成回答。

The MLX models used in this course have quantized weights. Dequantize each linear or embedding layer before loading it into tiny-llm by using the provided `quantize.dequantize_linear` function, then

store the readable Week 1 weight as BF16. Model activations and layer outputs should remain BF16. A readable attention or normalization expression may compute in FP32 for stability, but it must cast its model-facing result back to BF16.

本课程使用的 MLX 模型权重是量化过的。在把每个线性层或嵌入层加载进 tiny-llm 之前，请先使用已提供的 `quantize.dequantize_linear` 函数对其做反量化，然后把这份“可读的 Week 1 权重”存成 BF16。模型的激活值与各层输出应保持 BF16。为了数值稳定，可读性优先的注意力或归一化表达式可以在 FP32 下计算，但必须把最终对外输出的结果转回 BF16。

Pass `mask="causal"` to every Transformer block. For a one-token sequence the mask has no effect; for longer sequences, it prevents each position from attending to future tokens.

向每个 Transformer 块传入 `mask="causal"`。对单 token 序列而言该掩码不起作用；对更长的序列，它能阻止每个位置去 attend 未来的 token。

Run the tests for this task with:

用以下命令运行本任务的测试：

```
# Download each model you want to test. Missing models are skipped.
hf download Qwen/Qwen3-0.6B-MLX-4bit
hf download Qwen/Qwen3-1.7B-MLX-4bit
hf download Qwen/Qwen3-4B-MLX-4bit
pdm run test --week 1 --day 5 -- -k task_3
```

At the end of the day, you should be able to pass all tests of this day:

在今天结束时，你应该能通过当天的全部测试：

```
pdm run test --week 1 --day 5
```